Projet Python – Mastermind (version console)

Table des matières

1 - Présentation du projet Mastermind....2
1.1 - Objectif général....2
1.2 - Rappel des règles du Mastermind....2
1.3 - Progression pédagogique....2
2 - Partie 1 – Mastermind simplifié (obligatoire)....3
3 - Partie 2 – Mastermind complet (version avancée)....4
4 - Partie 3 – Mastermind inversé : le programme devine....5
4.1 - Que signifie « IA » dans ce projet ?....5
5 - Rendu et évaluation....6

1 - Présentation du projet Mastermind

1.1 - Objectif général

Ce projet a pour but de **reproduire le célèbre jeu de logique Mastermind** en version console. Il permet de **mobiliser progressivement les principales notions du langage Python** : structures conditionnelles, boucles, listes, fonctions, tirages aléatoires, gestion d'erreurs, etc.

Le projet est organisé en **trois grandes parties** :

1. **Version simplifiée** : apprentissage des bases du jeu.
2. **Version avancée** : respect des vraies règles du Mastermind.
3. **Version inversée** : l'ordinateur devient le joueur.

Chaque partie est elle-même découpée en **étapes numérotées**, chacune reprenant le code de la précédente.

1.2 - Rappel des règles du Mastermind

Le Mastermind oppose deux joueurs :

- **Le codeur** choisit une combinaison secrète de **4 pions colorés** (ou chiffres).
- **Le décodeur** tente de la retrouver en un nombre limité d'essais (souvent 10 ou 12).

Règles officielles :

- Chaque position contient **une couleur parmi 6** (rouge, bleu, jaune, vert, blanc, noir).
- Les **couleurs peuvent être répétées**.
- Après chaque tentative, le codeur indique :
  - 🔴 **Pion rouge (ou noir)** → bonne couleur, **bonne position**
  - ⚪ **Pion blanc** → bonne couleur, **mauvaise position**
- Le décodeur **gagne** s'il trouve la combinaison avant la fin des essais.

1.3 - Progression pédagogique

Chaque étape **reprend et améliore le programme précédent**.

L'étape *n* s'appuie sur le code de l'étape *n – 1* et ajoute de nouvelles fonctionnalités.

Les étudiants peuvent :

- **copier** leur code précédent, ou
- **le modifier directement** puis **enregistrer sous un nouveau nom**.

Ainsi, le projet forme une **construction progressive et cumulative** du jeu.

2 - Partie 1 – Mastermind simplifié (obligatoire)

Étape	Nom du script	Objectif principal	Notions clés	Contenu principal
Étape 1 – Génération du code secret	`s601_mastermind_tirage.py`	Créer la combinaison secrète	`random`, `list`, `join()`	Générer aléatoirement 4 chiffres distincts entre 1 et 6 à l'aide de `random`. Stocker la combinaison dans une liste et l'afficher temporairement pour vérification.
Étape 2 – Lecture de la proposition	`s602_mastermind_proposition.py`	Lire et valider la saisie utilisateur	`input()`, `try/except`, `while`	Demander une proposition à l'utilisateur (ex. "Entrez 4 chiffres entre 1 et 6 :"). Vérifier qu'elle contient 4 chiffres valides, sans doublons. Redemander tant que la saisie est incorrecte.
Étape 3 – Comparaison avec le code secret	`s603_mastermind_comparaison.py`	Tester la correspondance entre deux listes	`for`, `if` imbriqués	Comparer la proposition du joueur au code secret : compter les chiffres bien placés et mal placés, puis afficher le résultat ("→ 2 bien placés, 1 mal placé").
Étape 4 – Boucle de jeu complète	`s604_mastermind_jeu_complet.py`	Gérer les essais et la fin de partie	`while`, compteur	Répéter les essais jusqu'à trouver le code ou atteindre 10 tentatives. Afficher le nombre d'essais restants et un message de fin.
Étape 5 – Version finale simplifiée	`s605_mastermind_final.py`	Améliorer l'expérience utilisateur	messages, rejouer, affichage clair	Ajouter les messages de victoire/défaite et la possibilité de rejouer sans relancer le programme. Soigner la présentation à l'écran.

3 - Partie 2 – Mastermind complet (version avancée)

Étape	Nom du script	Objectif principal	Nouveaux éléments	Contenu principal
Étape 6 – Gestion des doublons	`s606_mastermind_doublons.py`	Gérer les chiffres répétés	logique plus fine	Autoriser des chiffres répétés dans le code (ex. `1 1 3 5`). Adapter la comparaison pour ne pas compter deux fois un même pion “mal placé”.
Étape 7 – Utilisation des couleurs (avec dictionnaire)	`s607_mastermind_couleurs.py`	Passer des chiffres aux couleurs réelles	`dict`, clés/valeurs, `random.choices()`	Remplacer les chiffres par des codes de couleurs stockés dans un dictionnaire : `{"re":"rouge","bu":"bleu", "ve":"vert","ja":"jaune", "no":"noir","bl":"blanc" }` Utiliser ces clés dans le code secret et afficher les couleurs en toutes lettres.
Étape 8 – Modularisation du code	`s608_mastermind_fonctions.py`	Structurer le programme	`def`, paramètres, retours	Créer des fonctions (`generer_code`, `lire_proposition`, `comparer`, `jouer_partie`). Rendre le code plus clair et réutilisable.
Étape 9 – Variante 2 joueurs	`s609_mastermind_pvp.py`	Interaction entre joueurs	saisie, effacement console	Le joueur 1 saisit le code secret (non affiché), le joueur 2 le devine. Possibilité d’échanger les rôles à la fin.

Règles de comparaison (version complète)

- Compter d’abord les **bien placés**, puis les **mal placés** parmi les symboles restants.
- Les positions exactes ne sont jamais indiquées.

4 - Partie 3 – Mastermind inversé : le programme devine

Étape	Nom du script	Objectif principal	Nouveaux éléments / algorithmes	Contenu principal
Étape 10 – Le joueur choisit un code	`s610_mastermind_ia.py`	Interaction avec le joueur	validation de saisie	Le joueur saisit son code secret (mêmes règles que précédemment). Le programme le stocke sans l'afficher.
Étape 11 – Tentatives automatiques	`s610_mastermind_ia.py`	Générer des propositions	`random`, boucles	Le programme propose des combinaisons aléatoires et affiche ses essais. Le joueur indique le nombre de pions bien/mal placés.
Étape 12 – Stratégie d'élimination	`s610_mastermind_ia.py`	Début d'intelligence artificielle	filtrage de listes	Le programme conserve toutes les combinaisons possibles et élimine celles incompatibles avec les résultats précédents.
Étape 13 – IA améliorée	`s610_mastermind_ia.py`	Recherche optimisée	raisonnement logique	Implémenter une stratégie efficace (ex. méthode de Knuth) pour réduire le nombre d'essais.
Étape 14 – Bilan et performances	`s610_mastermind_ia.py`	Analyse de résultats	statistiques, moyennes	Faire jouer plusieurs parties automatiquement, mesurer le nombre moyen d'essais et évaluer les performances.

4.1 - Que signifie « IA » dans ce projet ?

Dans ce contexte, le terme **IA (Intelligence Artificielle)** ne désigne **pas** une IA « qui apprend » comme dans le *machine learning* (aucune donnée n'est mémorisée ni entraînée).

Ici, **IA = Intelligence Algorithmique** :

le programme applique une **logique de raisonnement** pour deviner le code à partir des informations disponibles.

Trois niveaux d'« intelligence » dans cette partie

1. IA aléatoire (Étape 11) : le programme fait des essais au hasard.
2. IA logique (Étape 12) : il garde en mémoire les combinaisons possibles et élimine celles incompatibles avec les indices reçus.
3. IA optimisée (Étape 13) : il choisit la combinaison suivante en cherchant à réduire au maximum l'incertitude (méthode mathématique de Knuth).

Cette approche développe la compréhension des notions de :

- raisonnement par déduction,
- recherche par filtrage,
- efficacité algorithmique, tout en restant dans un cadre Python simple et ludique.

5 - Rendu et évaluation

- **Support :** Moodle
- **Format attendu :** fichiers `.py` nommés selon l'étape, avec en-tête complet :

```
# coding: utf-8
# Nom du fichier : s601_mastermind_tirage.py
# Auteur : Nom Prénom
# Date : octobre 2025
# Version : 1.0
# Objet : Projet Mastermind – Étape 1 – Génération du code secret
```

- **Critères d'évaluation :**
  - Fonctionnement conforme à l'énoncé
  - Code clair, commenté, structuré
  - Respect du format et de la logique progressive
  - Bonne gestion des saisies et erreurs
  - Bonus : modularisation, esthétique console, IA, variantes 2 joueurs